Lab 04

Process Management

Course: Operating System **Class: Computer Science IV**

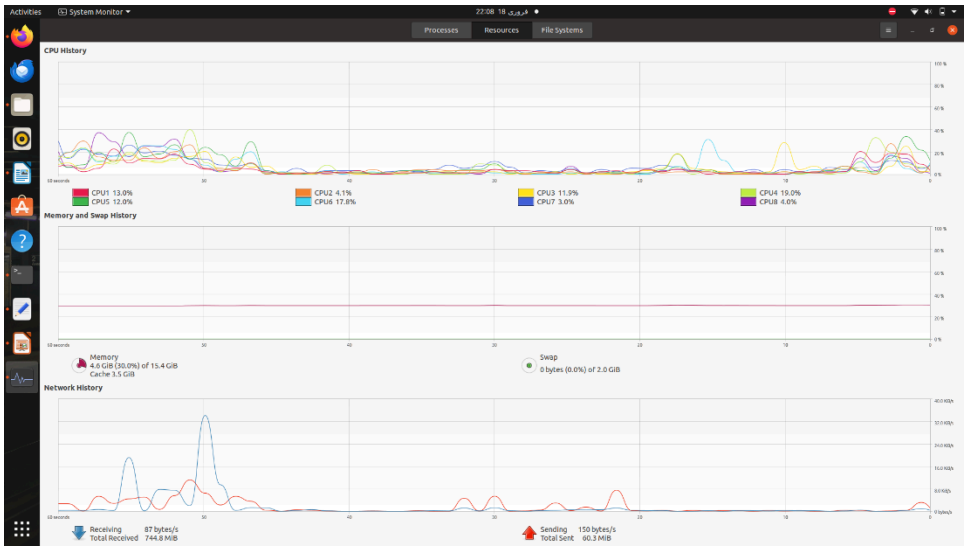
Instructor: Bilal Ahmed

Learning Objectives:

- Understand processes in practice.
- Learn about process types.
- Tools to monitor processes.
- Killing a process.
- Priority in Process

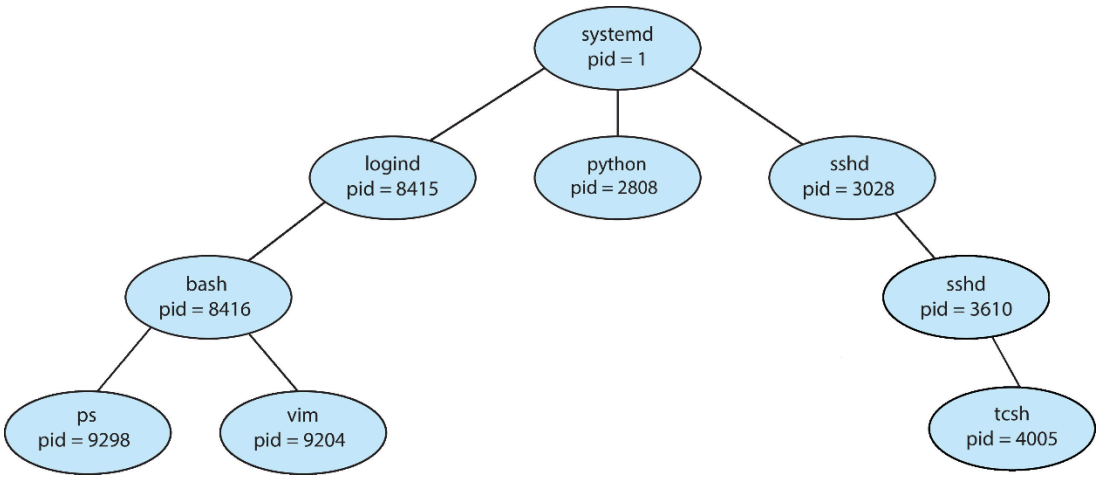
Process Managment: Refers to process monitoring, management, creation, termination and scheduling. It has great importance in operating systems as it allows you to have parallel processing and efficient use of shared resources. It is responsible for sharing resources such as CPU time, memory and peripheral devices efficiently between the multiple processes. Its basic aim is to maximize the CPU throughput and minimize the response time.

Process Managment in Linux: Linux provides various robust command line tools such as ps, top, htop, pstree, etc to monitor and manage the process running in operating system, whereas Windows and MacOS users often use GUI built-in tools like task manager and activity manager respectively.



PS – Process Status: It is one of the most basic tools of Linux to monitor the basic details of the process including its, process id, session id, terminal, command, memory and CPU usage and much more. Execute the ps to see processes running in current terminal.

Process ID (PID): A distinct identifier is assigned to each process, enabling its unique identification within the system. General tree of process of pids is given in following picture (taken from book).



Terminal Type (TTY): The type of terminal to which a user is currently logged in, specifying the communication interface for interaction.

CPU Time (TIME): The duration, measured in minutes and seconds, that a process has utilized the central processing unit (CPU) since its initiation.

Command Name (CMD): The label representing the name of the command that initiated the process, providing insight into the executed action.

Process States: In operating systems, process usually experiences some states from initiation to termination as illustrated below from the following figure from the book.

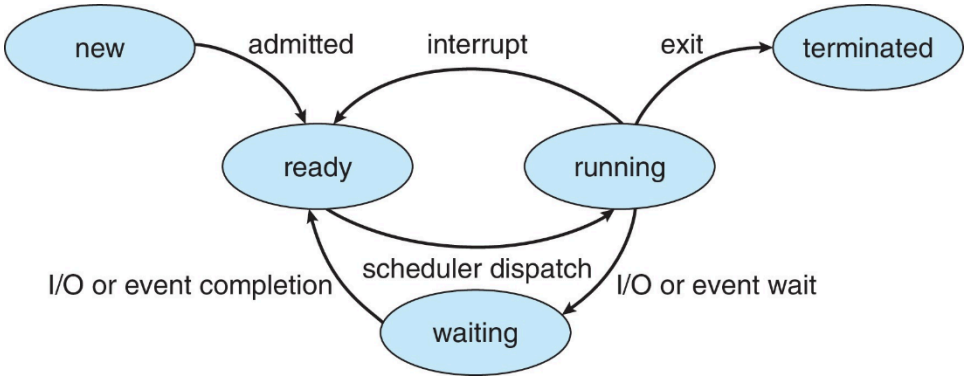
New: The process is being created

Running: Instructions are being executed

Waiting: The process is waiting for some event to occur

Ready: The process is waiting to be assigned to a processor

Terminated: The process has finished execution



Process States in Linux: Linux follows the following symbolic representation for process states.

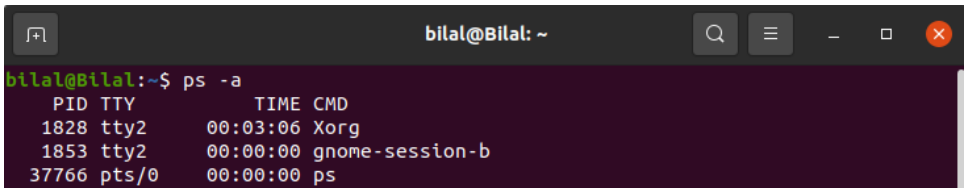
- **S**: Sleeping - Process is waiting for an event to complete.
- **R**: Running - Process is currently executing or ready to execute.
- **D**: Uninterruptible sleep - Process is waiting for a resource that cannot be interrupted, like a disk I/O operation.
- **T**: Stopped - Process has been halted, usually by receiving a signal, and can be restarted later.
- **Z**: Zombie - Process has completed execution, but its entry remains in the process table.
- l: Process is multithreaded.
- **s:** Process is a session leader.
- **+**: In foreground - Indicates the process is in the foreground process group.
- **<**: Job control signal - Process has been modified by a job control signal.
- **N**: Low-priority - Indicates a process started with a nice value higher than 0.

Ps a: This command will print all processes running by all users.

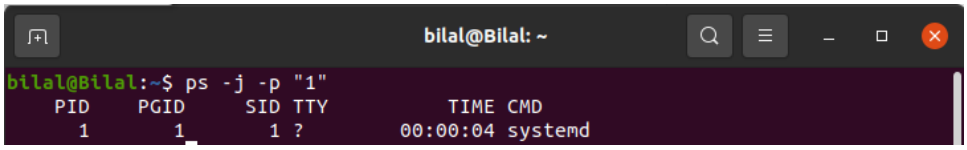
Explanation: Here STAT shows the process state as described for the above output as below.

- Ssl+ for PID 1403 indicates that the process is interruptible sleep (waiting for an event to complete), is a session leader (s), is in the foreground process group (+), and is also multi-threaded (l).
- Ss+ for PID 1406 indicates that the process is interruptible sleep, is a session leader (s), and is in the foreground process group (+).
- Ss for PID 40869 indicates that the process is interruptible sleep and is a session leader (s).
- R+ for PID 40968 indicates that the process is running and is in the foreground process group (+).

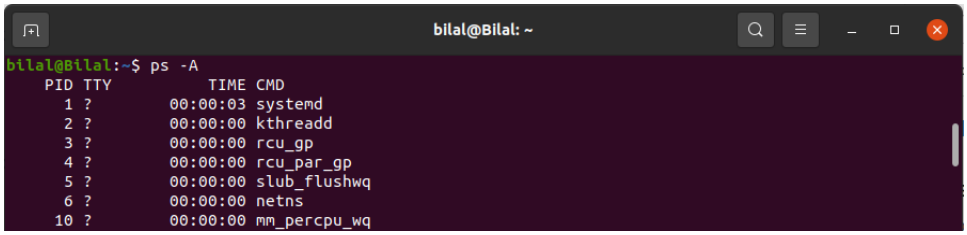
Ps - a: Used to display all processes by all users except session leaders and process associated terminals such as bash. Xorg and gnome are related with graphics and desktop.



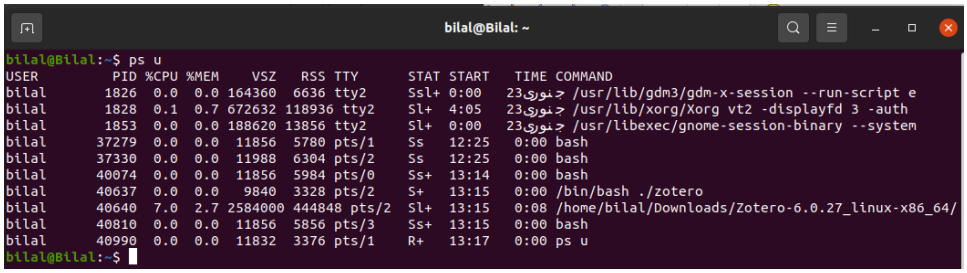
Systemd: Systemd is also a session leader, to check execute ps -j -p "1" to display the first process ID along with session ID.



Ps –A: Displays the entire processes running in the system.

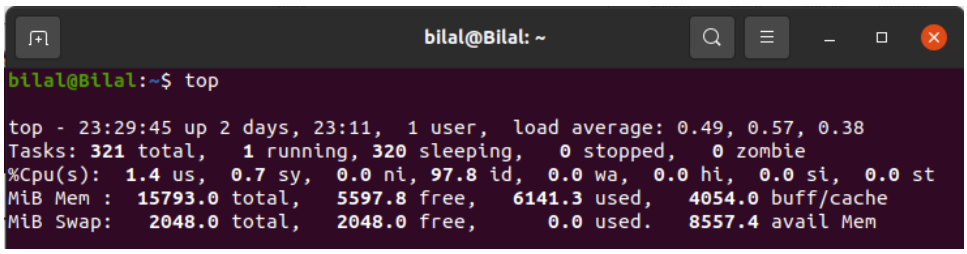


Ps u: Displays the CPU and memory usage.

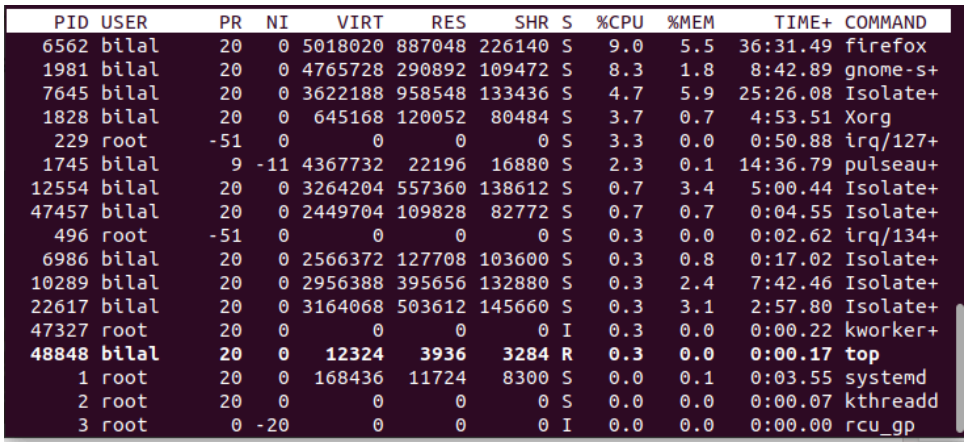


Top: Displays the Real-time processes with their PID, CPU, memory usage and other details. It shows the overall memory usage details at the top and individual process details.

Summary:

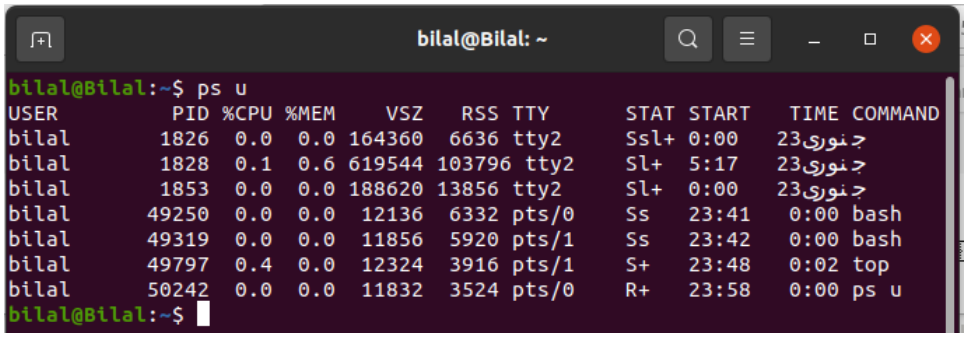


Individual Process Details:

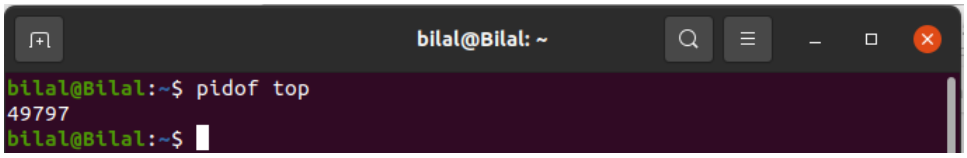


PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
6562	bilal	20	0	5018020	887048	226140	S	9.0	5.5	36:31.49	firefox
1981	bilal	20	0	4765728	290892	109472	S	8.3	1.8	8:42.89	gnome-s+
7645	bilal	20	0	3622188	958548	133436	S	4.7	5.9	25:26.08	Isolate+
1828	bilal	20	0	645168	120052	80484	S	3.7	0.7	4:53.51	Xorg
229	root	-51	0	0	0	0	S	3.3	0.0	0:50.88	irq/127+
1745	bilal	9	-11	4367732	22196	16880	S	2.3	0.1	14:36.79	pulseau+
12554	bilal	20	0	3264204	557360	138612	S	0.7	3.4	5:00.44	Isolate+
47457	bilal	20	0	2449704	109828	82772	S	0.7	0.7	0:04.55	Isolate+
496	root	-51	0	0	0	0	S	0.3	0.0	0:02.62	irq/134+
6986	bilal	20	0	2566372	127708	103600	S	0.3	0.8	0:17.02	Isolate+
10289	bilal	20	0	2956388	395656	132880	S	0.3	2.4	7:42.46	Isolate+
22617	bilal	20	0	3164068	503612	145660	S	0.3	3.1	2:57.80	Isolate+
47327	root	20	0	0	0	0	I	0.3	0.0	0:00.22	kworker+
48848	**bilal**	**20**	**0**	**12324**	**3936**	**3284**	**R**	**0.3**	**0.0**	**0:00.17**	**top**
1	root	20	0	168436	11724	8300	S	0.0	0.1	0:03.55	systemd
2	root	20	0	0	0	0	S	0.0	0.0	0:00.07	kthreadd
3	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	rcu_gp

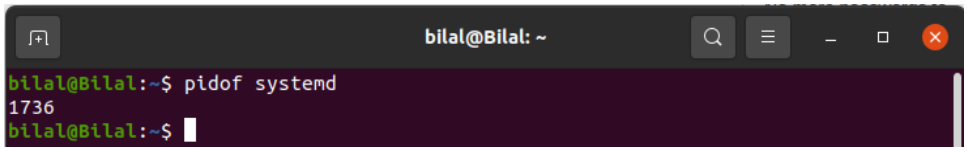
Pidof: Displays the pid of a process using its command. Open another terminal and execute **ps u** to see the **top** command running as below.



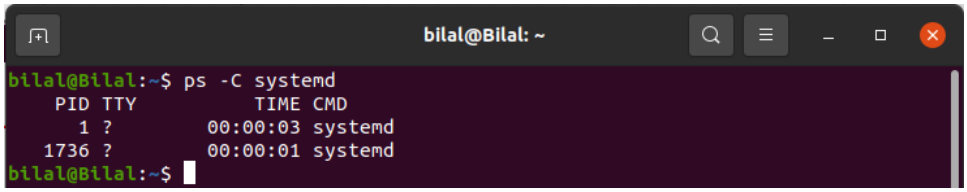
Above you can see the top command with pid of 49497. And we can see the pid of top using pidof command.



Systemd: Systemd is a Linux-specific system designed to operate within the Linux kernel. Serving as the initial process with a Process ID (PID) of 1, it is executed in user space as an integral part of the Linux startup sequence. Let's display the pid of systemd.



It may be a subprocess of systemd having different id of systemd, let's use ps –C systemd to print all possible systemd ids running in the system.

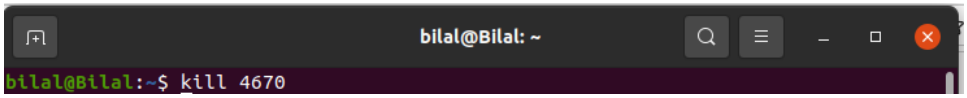


Killing the process: In operating system we also need terminate or kill the processes for some reason like unresponsiveness and huge memory use of a process. We can do it by killing it.

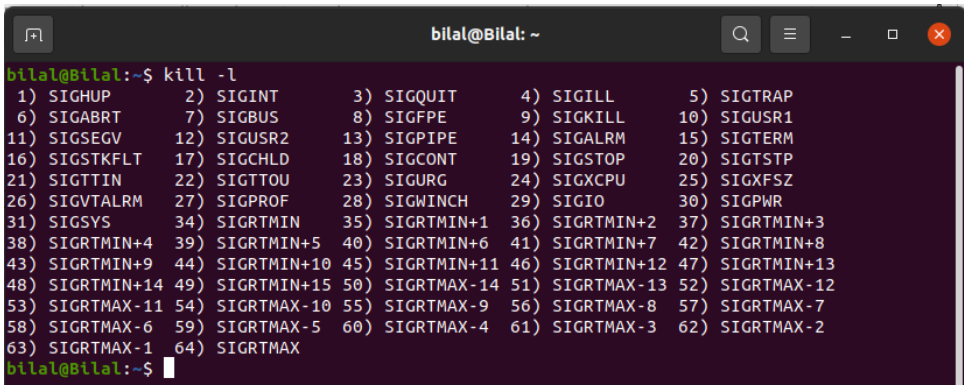
```
kill [signal] [PID]
```

Kill command is used to send a signal to a process, which can terminate it. Let's say there are two terminals (bash) that are open, and we want to close one of them. Below we can see we have two PIDs of bash and the one we are on has 8865.

Now let's terminate the bash having PID 4670 as below. After executing the following command, the other terminal will be terminated.



Signals: In Linux, signals are a fundamental inter-process communication mechanism. They are used to notify a process of a particular event, such as an interruption, termination, or user-defined event. We can print all signals along with their numbers by using ***kill –l*** command as below.



The most common signal used to terminate a process is SIGTERM (signal number 15) which is by default. Some of the most commons are given below.

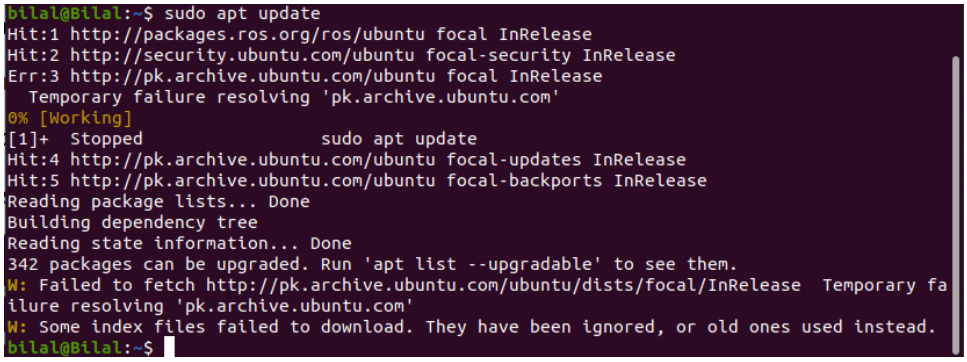
SIGTERM (15): Request to terminate a process.

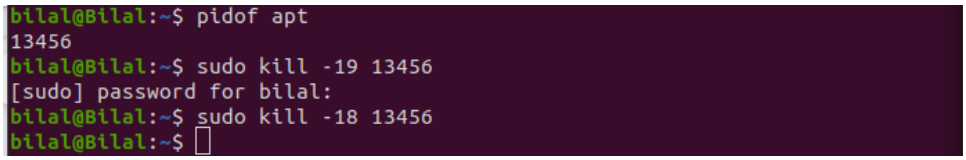
SIGKILL (9): Terminate the process immediately like End Task in windows.

SIGSTOP (19): Pause a process.

SIGCONT (18): Resume a process.

Example 1: Below you can see we initiated the update process using apt and checked its PID in the second terminal. Then we stopped (paused) it using signal 19 and resumed it using signal 18. As in the first we can witness the whole process.





Example 2: Open a terminal, note its pid, pause and resume it from another terminal, and see the behavior of the first terminal opened.

Killing process by name: We can kill the process using its name using ***killall*** command as in below given example we opened multiple terminals and closed all using single entity (its name).

Priority in Operating System: In any given moment, a Linux system hosts numerous processes, many initiated by the operating system itself, while others are generated by the logged-in user. Each process is assigned a priority by the system, dictating its execution speed. Generally, processes with higher priorities are executed before those with lower priorities. Priorities are 0 to 139 in which 0 to 99 for real-time and 100 to 139 for users.

Commands: Linux offers the 'nice' and 'renice' commands to modify a process's priority, thereby influencing its execution speed within the system.

Priority Types: Linux implements two types of priorities as given below.

Static Priority Range (NI ~ Niceness) (NI): Spans from -20 to 19, indicating the "niceness" of a process. Lower values signify higher priority, with 0 being the default. Processes with lower niceness values receive more CPU time, while those with higher niceness values receive fewer resources.

Dynamic Priority Range (PRI ~ Priority) (PRI): The priority value is the process's actual priority which is used by the Linux kernel to schedule a task. Ranges from 0 to 99 and is used for real-time tasks requiring immediate and predictable responses. Processes with higher dynamic priorities are scheduled with preference over those with lower priorities. The lower the value the higher the priority.

Example:

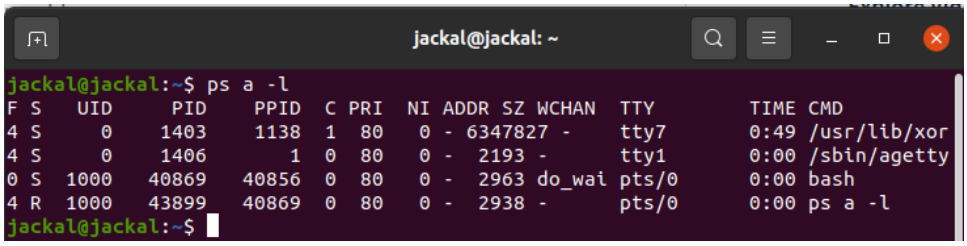
Static Priority Range (Niceness):
Suppose we have two processes, A and B, running on a Linux system. Process A has a nice value of -10, while process B has a nice value of 10.

Process A (nice value: -10) is considered to have a higher priority because it has a lower niceness value. It will receive more CPU time compared to other processes with higher niceness values.
Process B (nice value: 10) is considered to have a lower priority because it has a higher niceness value. It will receive fewer CPU resources compared to other processes with lower niceness values.

Dynamic Priority Range (Real-time):
Let's say we have a real-time task, such as audio processing or network packet handling, running on the system with a dynamic priority of 90.

The real-time task with a dynamic priority of 90 has the highest priority among all processes on the system. It is scheduled preemptively, meaning it can interrupt lower-priority processes to execute immediately. This ensures that the real-time task receives immediate and predictable responses from the system, crucial for tasks like audio processing or network packet handling.

Checking the priority: To check priority we can use ***ps –l*** command as given below.

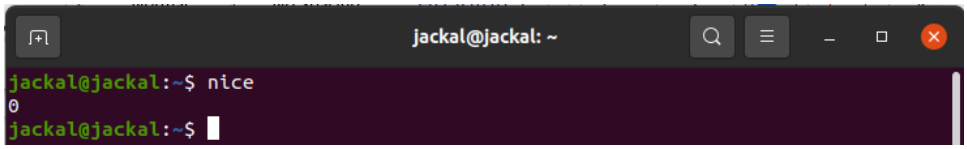


Explanation:

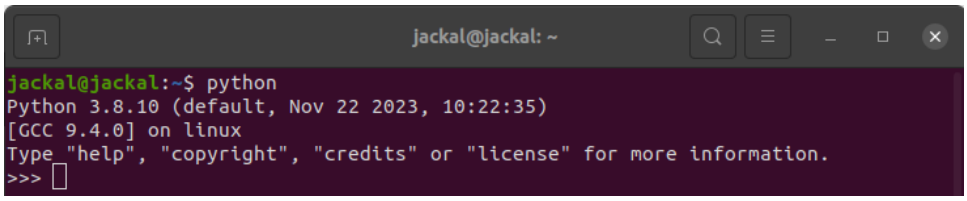
- **S**: The process state. This column indicates the current state of the process, such as running (R), sleeping (S), or waiting for input/output (D).
- **PRI**: The priority of the process. This column indicates the priority level assigned to the process by the system.

- **NI**: The nice value of the process. This column displays the "niceness" value of the process, which influences its priority.

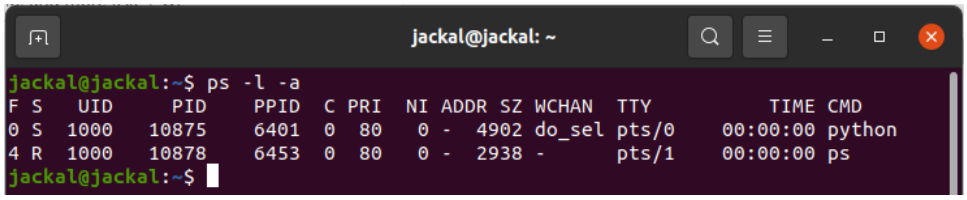
Nice: Executing nice command without any arguments, it sets the process's static priority ID to 0.



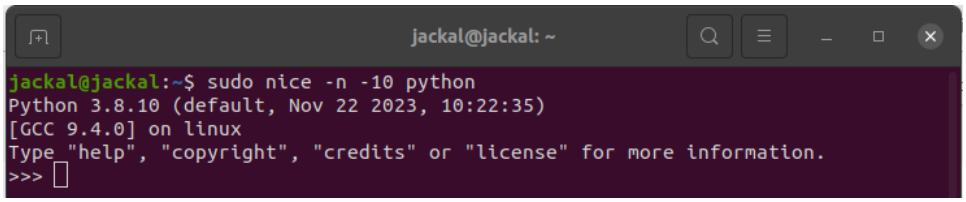
Example – Step 1: Launch the python program as follows.



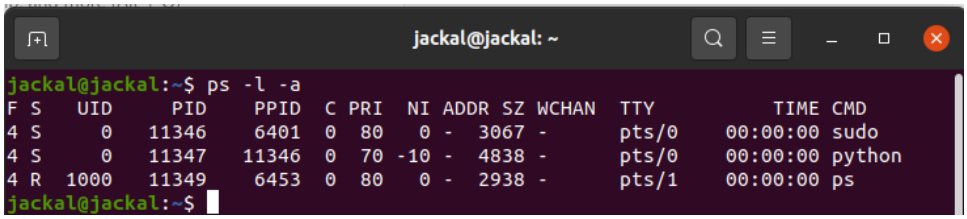
Step 2: Check the nice value.



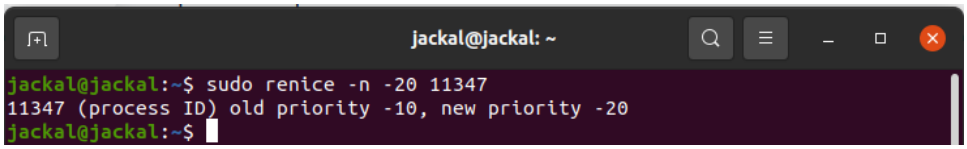
Step 3: Close the Python and launch with desired nice value with command ***nice -n -20 <command>*** as sudo user***.***



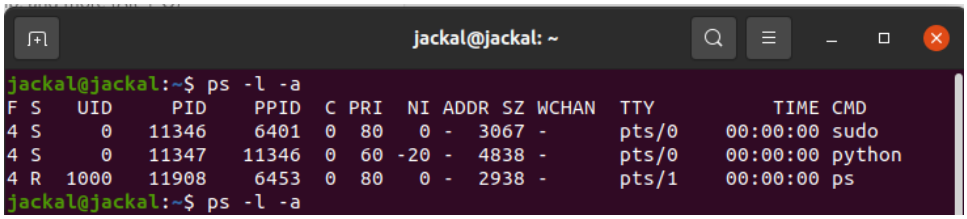
Step 4: Check the nice value now.



Changing Niceness of running process: Renice can be used to change the niceness value of a running process with command renice –n <PID> as sudo user.



Output:



Exercise:

Task 1: Explore the following commands with examples.

- Pstree
- Lsof
- Watch
- Free
- Df
- Strace

Task 2: Write a simple script.sh file. Print some information, add delay (sleep) and track the process and its PID of script and sleep process using any tool.

Task 3: Write a simple python or C code, add delay, kill it before it gets completed and report the process.

Task 4: Write one simple script file and python file, check their NI and PRI, change them using nice and renice and report the process.